
Trabalho Prático (10.0 Pts.) – Entrega 02/06/2026

Instruções que DEVEM ser seguidas:

- Esta atividade prática deve ser postada na data de entrega acima.
- Deve ser realizado com a máxima atenção e seguindo rigorosamente todas as instruções.
- Este trabalho deverá ser realizado em grupo com, no máximo, três (3) integrantes.
- As implementações deverão ser realizadas em uma única linguagem de programação escolhida pelo grupo, ou seja, não serão avaliadas duas ou mais implementações de código diferentes por grupo.
- Os códigos deverão ser reproduzíveis, isto é, cada código será executado e avaliado. Caso um determinado código execute parcialmente ou não execute, haverá penalizações ou mesmo zerado.
- A entrega deverá ser feita no ambiente **Moodle** por meio de um arquivo único compactado **ZIP** ou **RAR**.
- O arquivo compactado deve conter todos os respectivos arquivos do relatório técnico, ou seja, arquivo DOCX ou PDF, códigos da implementação e demais arquivos para o devido funcionamento.
- A legibilidade e objetividade são requisitos básicos para serem avaliadas adequadamente.
- Qualquer indício de cópia de questões ou do arquivo entregue, será caracterizada como COLA, e a nota será zerada para todos os envolvidos.

Contextualização do trabalho

A disciplina Teoria de Grafos, como uma ferramenta de modelagem de problemas do mundo real, conta com diversos algoritmos que visam solucionar diferentes domínios de problemas.

No entanto, é preciso compreender o domínio a ser modelado e, somente então, desenvolver uma modelagem robusta do problema com uma solução coerente, confiável e eficiente. Tais requisitos são cruciais, principalmente para problemas possuem um alto nível de complexidade, seja no nível de desenvolvimento do algoritmo, seja no custo computacional (tempo e/ou espaço) do mesmo.

O objetivo deste trabalho prático (TP) é realizar a implementação e análise dos algoritmos clássicos de caminho mínimo, como o algoritmo de Dijkstra e Bellman-Ford.

O aluno (grupo) deve compreender a fundamentação teórica do artigo "*Breaking the Sorting Barrier for Directed Single-Source Shortest*"¹ de 2025 (<https://dl.acm.org/doi/10.1145/3717823.3718179>), por meio da leitura e compreensão, buscando a solução dos problemas apresentados, via codificação de cada etapa.

O TP visa desenvolver as habilidades de experimentação e validação de resultados teóricos e práticos.

PARTE 1 - Algoritmos clássicos e comparação experimental (60%)

O artigo de Duan et al. (2025) [pág. 2, seção "Technical Overview"], descreve os dois paradigmas tradicionais para o problema de caminhos mínimos com fonte única (SSSP), onde "*There are two traditional algorithms for solving the single-source shortest path problem: - Dijkstra's algorithm [Dij59]: via a priority queue... - Bellman-Ford algorithm [Bel58]: based on dynamic programming...*".

1.1 Implementação do algoritmo de Dijkstra

- Estrutura de dados: Heap binário (*heapq*) [pág. 9, algoritmo 2 (BaseCase)].
 - Grafos: Direcionados, pesos não-negativos.
- Saída obrigatória:
 - Vetor de distâncias mínimas do vértice fonte para todos os vértices [pag.2].
 - Vetor de predecessores (para reconstrução dos caminhos).
 - Tempo de execução (em segundos).

1.2 Implementação do algoritmo Bellman-Ford

- Grafos: Direcionados, pesos não-negativos (para comparação justa).
- Complexidade: $O(m \cdot n)$ [pág. 2, visando a expansão da estratégia de Dijkstra].
- Deve detectar ciclos negativos (embora não existam nos grafos de teste).
- Saída: Mesmo formato do Dijkstra.

1.3 Estudo Experimental Comparativo por meio de um Gerador de Grafos:

- Implementar um gerador que produza grafos em um arquivo no formato txt.

```
# Formato do arquivo, onde, n = número de vértices, e = número de arestas, s = vértice inicial, u =
vértice com aresta, v = vértice com aresta, w = peso da aresta
n e
u v w
u v w
...
s
```

- Gere dois tipos de grafos:

Grafos esparsos	Árvores + arestas extras	$e = 2n, 3n, 4n$
Grafos densos	Quase completos	$e = n(n-1)/4, n(n-1)/2$

- Tamanhos de teste:

$n = \{100, 500, 1000, 5000, 10000\}$
$e = \text{conforme o tipo de grafo}$

Critérios de Avaliação da Etapa 1

Critério	Peso	Descrição
Corretude	40%	Resultados idênticos à verificação
Eficiência	20%	Implementação sem ineficiências óbvias
Experimentos	30%	Todos os tamanhos/famílias reportados
Relatório	10%	Análise crítica dos resultados

PARTE 2 - Implementação do algoritmo *find_pivots* (40%)

A principal contribuição teórica do artigo está na capacidade de reduzir a fronteira do Dijkstra. O Lema 3.2 [pág. 5] afirma: “*The idea of finding pivots is as follows: we perform relaxation for k steps... the number of large shortest path trees from S , consisting of at least k vertices and rooted at vertices in S , is at most $|\tilde{U}|/k$* ”.

2.1 Implementar o Algoritmo 1 (FindPivots, pág. 6) e verificar experimentalmente o Lema 3.2

- Considere o protótipo em alto nível do algoritmo *find_pivots*:

```
def find_pivots(B: float, S: List[int], graph: Graph, k: int) -> Tuple[Set[int], Set[int]]:
    """
    Implementação do Algoritmo 1 do artigo.
    Args:
        B: Limite superior de distância
        S: Conjunto de vértices fonte (fronteira)
        graph: Grafo com distâncias estimadas ( $\hat{d}$ )
        k: Parâmetro de profundidade ( $k = \lfloor \log^{(1/3)}(n) \rfloor$  no artigo)
    Returns:
        P: Conjunto de pivôs
        W: Conjunto de vértices visitados/completados
    """
```

- Requisitos:
 - Manter o array global $\hat{d}$ (distâncias estimadas).
 - Executar exatamente k iterações de relaxamento.
 - Construir a floresta F (arestas relaxadas).
 - Identificar raízes de árvores com $\geq k$ vértices.
 - Respeitar a condição de parada antecipada se $|W| > k|S|$.

2.2 Verificação Experimental do Lema 3.2

- Deve-se verificar as seguintes questões:
 - Tamanho dos pivôs: $|P| \leq |W|/k$.
 - Cobertura: Todo vértice x em $\tilde{U}$ (distância $< B$ que visita S) ou está em W ou tem caminho via P .
 - Complexidade: $O(k \cdot |W|) \leq O(\min(k^2|S|, k|\tilde{U}|))$.

Procedimento experimental:

- Para cada grafo de teste da parte 1:
 - Escolha um vértice fonte s .
 - Execute o algoritmo de Dijkstra parcial até processar $\forall n$ vértices.
 - Capture o conjunto S (fronteira) e o valor B (distância do último processado).
 - Execute *find_pivots*(B, S, k) com $k = 2, 4, 8, 16$.
- Coletar as seguintes métricas:
 - $|S|, |W|, |P|, |\tilde{U}|$.
 - Razão $|P| / (|W|/k)$.
 - Tempo de execução vs. $k \cdot |W|$.
 - Porcentagem de vértices cobertos por P .
- Visualização dos dados (obrigatório):
 - Histograma: distribuição de $|P| / (|W|/k)$ para múltiplos experimentos.
 - Gráfico: $k \times |P|$ (esperado: decrescimento).
 - Gráfico: $|S| \times |P|$ (esperado: sublinear).

2.3 Aplicação: Pré-processamento para o algoritmo de Dijkstra

- Utilize o FindPivots como pré-processamento para o Dijkstra.
- Algoritmo proposto:
 - Execute FindPivots(B, S, k).
 - Use o conjunto P como "super-fontes".
 - Execute Dijkstra a partir de cada $p \in P$.
 - Combine os resultados.
- Perguntas a responder:
 - Há ganho de desempenho? Para quais valores de k?
 - O ganho é maior em grafos esparsos ou densos?
 - A perda de precisão (caminhos não-ótimos) é zero?

ROTEIRO DE EXPERIMENTOS

- Experimento 1: Escalabilidade

n	e (esparso)	e (denso)
100	200	2.500
500	1.000	62.500
1.000	2.000	250.000

Experimento 2: Sensibilidade ao parâmetro k (PARTE 2)

k = {2, 4, 6, 8, 10, 12, 14, 16}
n fixo = 1000, e = 2000 (esparso)

ESTRUTURA DO RELATÓRIO - Seções obrigatórias:

1 Introdução (1 página) 1.2 O problema SSSP 1.3 Contextualização do artigo de Duan et al. (2025)	3 Experimentos e Resultados (3~4 páginas) 3.1 Metodologia (hardware, software, medição) 3.2 Tabelas e gráficos 3.3 Análise dos resultados
2 Implementação (2~3 páginas) 2.1 Decisões de projeto 2.2 Estruturas de dados utilizadas 2.3 Pseudocódigo comentado das funções principais	4 Discussão (1~2 páginas) 4.1 O que os resultados dizem sobre o artigo? 4.2 Dificuldades encontradas 4.3 Limitações da implementação
	5 Conclusão (1 página)

ORIENTAÇÕES FINAIS

- Trabalho em grupo: Máximo 3 alunos
- Reprodutibilidade: Todos os experimentos devem ser executáveis com um único script.
- Referencie o artigo supracitado, além dos artigos e/ou livros complementares.
- Originalidade: Os códigos serão comparados por similaridade na codificação.

Bom trabalho!